

Four Strongly Typed CLI Tools from a Minimal DSL

Interpreter specialization using dependent indices

16 August 2026

Abstract

This note describes a small C++23 command DSL implementing literal `grep`, `head`, `cut`, and `wc -l`. Programs are indexed by input and output object types. A generic interpreter executes the DSL, while specialization captures static command parameters and produces residual host functions. The construction is a practical partial-evaluation analogue of normalization followed by interpretation.

1 Typed object language

The object-level types are `Text`, `Lines`, and `Count`. A program has the indexed form `Program(I, O)`. The tools have signatures

$$\text{grep, head, cut} : \text{Program}(\text{Text}, \text{Text}), \quad \text{wcLines} : \text{Program}(\text{Text}, \text{Count}).$$

The node variant is selected by (I, O) , preventing a count-producing node from being used where text is required.

2 Interpreter and specialization

The generic evaluator has type

$$\text{interpret} : \text{Program}(I, O) \rightarrow I \rightarrow O.$$

The specialization pass performs

$$\text{specialize}(p) : I \rightarrow O$$

by matching once on the normalized command constructor and capturing its static pattern, line count, delimiter, or field number. Runtime input is processed by the residual closure, with command dispatch removed.

$$\text{normalize}(p) : \text{Program}(I, O).$$

Thus normalization preserves the endpoint type, just as reification preserves the type of a dependently typed term.

3 The four tools

Literal `grep` has type `Text` $\rightarrow$ `Text` and retains lines containing a static pattern. `Head` has type `Text` $\rightarrow$ `Text` and retains at most the first static number of lines. `Cut` has type `Text` $\rightarrow$ `Text` and selects a static field under a static delimiter. `Wc lines` has type `Text` $\rightarrow$ `Count`, explicitly recording that its result is no longer text.

4 Executable evidence

The implementation is in `cli_dsl.cpp`. It was compiled with:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 cli_dsl.cpp -o cli_dsl
./cli_dsl
```

The interpreter and specialized residual function are compared on the same sample input. The output is:

```
grep: PASS
head: PASS
cut: PASS
wc -l: PASS
all specialized CLI tools: PASS
```

5 Relation to dependent typing

A full object-language version would replace the C++ index with an indexed syntax family $\text{Cmd}(I, O)$ and a typed evaluator $\text{run} : \text{Cmd}(I, O) \rightarrow I \rightarrow O$. Static specialization would then be a typed transformation preserving (I, O) . The present prototype demonstrates the same boundary: static syntax is normalized first, and only the residual program consumes dynamic input.

6 Limitations

The prototype does not yet model filesystem effects, process exit codes, lazy streaming, shell quoting, or indexed pipeline composition. Those features should introduce explicit effect and resource indices rather than weakening the input/output type boundary.